MODULE 15 · DOMAIN 2 - APPLICATIONS AND INTEGRATION

Tests, Version Control, Review and Refactoring

The testing pyramid for Claude features, evals and graders, prompts under version control, reviewing a Claude change, and refactoring small and large.

DOMAIN 2 of 8 EXAM WEIGHT 33.1% MODULE 15 of 36

admin@sthithapragna.com

Test a car in three ways: the parts on the bench, the engine on the stand, then the road test.

The testing pyramid for a Claude feature

LAYER	WHAT IT TESTS, AND HOW	WHEN
Unit	Your own code, with no API call	Every commit
Integration	The API contract, with recorded responses	Every merge
Evals	Model behaviour, with a labelled set and graders	Prompt or model change
End-to-end	A few critical journeys, in staging	Before release

THE TIEBREAKER

Varied output goes in evals, judged by pass rates. Your own code is judged exactly.

Designing evals

- **Be task-specific:** "mirror your real-world task distribution. Don't forget to factor in edge cases!"
- **Automate when possible:** multiple-choice, string match, code-graded, LLM-graded.
- **Volume over quality:** more questions with automated grading beat fewer hand-graded ones.
- **Edge cases:** missing or irrelevant input, very long input, harmful input, ambiguous cases.
- **Claude can help:** generate more test cases from a baseline set.

> ON THE EXAM
>
> Evals start from the success criteria in module 9. Each criterion needs at least one way to measure it.

Three ways to grade

METHOD	STRENGTH	USE IT FOR
Code-based	Fastest, most reliable; lacks nuance	Exact labels, required fields
LLM-based	Fast, handles judgement; validate first	Tone, completeness, rubrics
Human	Highest quality; slow and expensive	Calibrating graders, ambiguous cases

THE TIEBREAKER

Anthropic: choose "the fastest, most reliable, most scalable method". Human grading: "Avoid if possible."

A model grader with a rubric

```
1  RUBRIC = ("Correct only if the reply names the trade, promises no repair date, "
2            "and gives the emergency instruction when urgency is emergency.")
3
4  def grade(reply, ticket):
5      result = client.messages.create(
6          model=GRADER_MODEL, max_tokens=1024,      # not the model being graded
7          messages=[{"role": "user", "content":
8              f"<rubric>{RUBRIC}</rubric><ticket>{ticket}</ticket><reply>{reply}</reply>"
9              "Output correct or incorrect in <result> tags."}])
10     return parse_result(result)                   # "correct" or "incorrect"
11
12 assert pass_rate(grade_all(TRIAGE_SET)) >= 0.95   # threshold from requirements
```

1-2 A detailed, specific rubric, not "is it good?".

9, 12 A specific output, and a threshold agreed in advance.

THE TAKEAWAY

Validate the grader against human labels on a sample before trusting its pass rates.

Regression sets

- **Every bug becomes a test case.** The tenant who said "gas" in Welsh stays in the set forever.
- **Run on every prompt, model or tool change,** and compare with the last release, not just a threshold.
- **Keep a held-out set** nobody tunes the prompt against, to catch overfitting.
- **Record the run:** prompt version, model ID, pass rates, failures.
- **One good run proves little.** Varied output needs enough cases, and repeats on the hard ones.

> THE TAKEAWAY
>
> A regression set is the memory of every mistake you have already fixed.

Prompts under version control

PROMPT AS A STRING

Edited wherever it lives

- Pasted inline in several files
- Changed in a console, unreviewed
- No record of what ran when
- No way to roll back

PROMPT AS CODE

Versioned and reviewed

- One file in the repository, with a version
- Pull request, review, eval run in CI
- prompt_version logged on every call
- Roll back like any release

ON THE EXAM

Prompts, model IDs and parameters are configuration. They get version control, review and a deploy pipeline.

Reviewing a Claude change

- **Prompt and model from configuration,** never hard-coded.
- **Errors and stop reasons handled:** retries, refusals, max_tokens, fallbacks.
- **Outputs validated** before anything acts on them.
- **Secrets from the secrets manager;** no keys in code, logs or prompts.
- **Tests and evals** included, with results attached to the pull request.
- **Logging:** request ID, prompt version, usage. No personal data beyond what is needed.

THE TAKEAWAY

Claude can review code too, but a person still owns the approval.

Refactoring: small or large?

SMALL

One focused pass

- Contained: one module or service
- Tests already cover it
- One reviewed change
- Claude Code can draft it quickly

LARGE

Incremental, tests first

- Many services or call sites
- Characterisation tests first
- Small reviewed steps behind a stable interface
- Each step verified before the next

THE TIEBREAKER

Rule six: change in small, tested steps. The bigger the change, the smaller each step.

Copperlet: the prompt in eleven places

BEFORE

Eleven copies

- Triage prompt inline in 11 files, 3 services
- Model ID hard-coded beside it
- Nobody knew which copy was current
- No test caught a drift

AFTER

One source

- Tests first: the 200-ticket set and fixtures
- Each service moved to the prompts repo, one at a time
- Model ID from the client module's config
- CI fails on any prompt text outside the repo

SETTLED

Quirk 1 settled: one prompt, one version, one place, and a check that keeps it that way.

A team edits its support assistant's prompt every few weeks. Each time, a developer tries a handful of questions by hand before release. Twice this quarter, a change fixed one problem and quietly broke another.

The team wants to catch these before customers do.

What should the team add?

A. An automated eval on a labelled set with edge cases, run in CI on every change

B. A feedback button, so that customers can report bad answers after release

C. More instructions in the prompt, covering every problem fixed so far

D. A longer manual checklist, so the developer tries more questions before release

◆ PAUSE AND DECIDE

What should the team add?

A. An automated eval on a labelled set with edge cases, run in CI on every change
Catches regressions automatically, every time.

B. A feedback button, so that customers can report bad answers after release
Useful, but customers find the bug first.

C. More instructions in the prompt, covering every problem fixed so far
More change, still untested.

D. A longer manual checklist, so the developer tries more questions before release
Slow, and still a handful.

THE SIGNAL

"Fixed one problem and broke another" is the signal for a regression eval in CI.

A team must grade 2,000 generated summaries per release for completeness and tone. Human grading takes a week and blocks every release. The wording of good summaries varies widely.

The team wants fast, reliable grading.

Which approach fits best?

A Exact string matching against one golden summary per input

B Keep human grading for every summary, and release less often

C Ask the summarising model to rate its own outputs, without a rubric

D A model grader with a detailed rubric and specific outputs, checked against human grades on a sample first

◆ PAUSE AND DECIDE

Which approach fits best?

A Exact string matching against one golden summary per input

Good wording varies. Everything would fail.

B Keep human grading for every summary, and release less often

Slow and expensive. Avoid if possible.

C Ask the summarising model to rate its own outputs, without a rubric

No rubric, and grading its own work.

D A model grader with a detailed rubric and specific outputs, checked against human grades on a sample first

Fast, handles judgement, validated.

THE SIGNAL

Judgement at volume: a model grader with a rubric, validated against people before it is trusted.

A team's CI runs every test against the live Claude API. Tests for the response parser and prompt builder fail at random and cost money, and nobody can tell whether a failure is a code bug or a change in model output.

The team wants a reliable test suite.

Which two changes should they make? Choose two.

A Delete the tests that fail at random

B Retry each failed test until it passes

C Test the parser and prompt builder as unit tests with recorded responses, without API calls

D Keep checks on live model behaviour in a separate eval suite, judged by pass-rate thresholds

E Set the temperature to zero so that live outputs are identical

◆ PAUSE AND DECIDE

Which two changes should they make? Choose two.

A Delete the tests that fail at random

Loses the coverage along with the noise.

B Retry each failed test until it passes

Hides real failures.

C Test the parser and prompt builder as unit tests with recorded responses, without API calls

Your code, tested exactly and cheaply.

D Keep checks on live model behaviour in a separate eval suite, judged by pass-rate thresholds

Varied output, judged as a rate.

E Set the temperature to zero so that live outputs are identical

Not accepted on current models, and never identical.

THE SIGNAL

Split the layers: deterministic tests for your code, evals with thresholds for the model.

> A company's prompts are edited directly in a production configuration console. After an incident, nobody can say what the prompt said last Tuesday, or who changed it.
>
> The team wants changes to be traceable and reversible.

What is the best practice?

A Restrict console access to a single administrator

B Keep prompts in version control, changed by reviewed pull requests, version logged per call

C Keep a changelog page in the team wiki, updated by whoever edits a prompt

D Export the console settings to a dated backup file every week

◆ PAUSE AND DECIDE

What is the best practice?

A Restrict console access to a single administrator

Fewer editors, still no history or review.

B Keep prompts in version control, changed by reviewed pull requests, version logged per call

History, review, rollback and traceability.

C Keep a changelog page in the team wiki, updated by whoever edits a prompt

Depends on memory, and drifts.

D Export the console settings to a dated backup file every week

A week of changes can vanish.

THE SIGNAL

"What did it say last Tuesday, and who changed it?" Version control answers both.

> A pull request adds a Claude summarisation feature. The model ID and prompt are hard-coded, there is no handling for errors or stop reasons, an API key sits in a source file, and there are no tests.
>
> It worked well in the demo.

How should the reviewer respond?

A Request changes: config, secrets manager, error and stop-reason handling, tests, logging

B Approve, because the demo went well and the rest can be hardened in a later release

C Request only that the key be moved, since the other points are matters of style

D Approve, and ask Claude to fix any problems once the feature is in production

◆ PAUSE AND DECIDE

How should the reviewer respond?

A. Request changes: config, secrets manager, error and stop-reason handling, tests, logging
The complete answer: every gap closed.

B. Approve, because the demo went well and the rest can be hardened in a later release
A demo is not production.

C. Request only that the key be moved, since the other points are matters of style
The other gaps are failures waiting.

D. Approve, and ask Claude to fix any problems once the feature is in production
Ships known gaps to customers.

THE SIGNAL

Rule four, the complete answer: the option that closes every gap the scenario lists.

A company wants to replace a home-grown HTTP wrapper with the official SDK across 30 services. One engineer proposes using Claude Code to rewrite everything in a single weekend, in one large pull request.

Test coverage around the wrapper is thin.

What is the best approach?

A. Accept the single large pull request, since Claude Code can make the edits consistently

B. Freeze all feature work for a month, and rewrite every service from scratch

C. Leave the wrapper in place, since migrations are risky

D. Characterisation tests first, then migrate one service at a time, verifying each step

◆ PAUSE AND DECIDE

What is the best approach?

A Accept the single large pull request, since Claude Code can make the edits consistently

Unreviewable, and nothing to catch errors.

B Freeze all feature work for a month, and rewrite every service from scratch

Maximum risk, maximum cost.

C Leave the wrapper in place, since migrations are risky

Avoids the work the business wants.

D Characterisation tests first, then migrate one service at a time, verifying each step

Small, tested, reversible steps.

THE SIGNAL

"Thirty services" plus "thin coverage": tests first, then small steps. Claude Code drafts each step.

Module 15 in five lines

- **The pyramid:** unit tests for your code, integration, evals for the model, a few end-to-end.
- **Evals:** task-specific, automated, high volume, edge cases included.
- **Grade with code** where you can, **a validated model grader** for judgement.
- **Prompts are code:** versioned, reviewed, deployed, logged by version.
- **Refactor small in one pass;** large with tests first, in small steps.

COMING UP

Next: configuration management - CLAUDE.md, settings, pinning, versioning and plugins.